PSEUDOCODE: STUDENT RECORD SEARCH SYSTEM

```
ALGORITHM STUDENT-RECORD-SEARCH
INPUT:
    A = Array of student registration numbers
    n = Number of student records
    key = Registration number to search
OUTPUT:
    Search results using three algorithms
BEGIN
    SET A ← [101, 205, 309, 412, 518, 624, 735]
    SET n ← LENGTH(A)
    INPUT key
    // 1. LINEAR SEARCH
    SET linearResult ← NOT FOUND
    FOR i ← 0 TO n - 1 DO
        IF A[i] = key THEN
            SET linearResult ← i
            BREAK
        END IF
    END FOR
    // 2. RECURSIVE BINARY SEARCH
    SET low ← 0
    SET high ← n - 1
```

```
SET binaryResult ← BINARY-SEARCH(A, low, high, key)
// 3. INITIALIZE HASH TABLE
SET SIZE ← 101
FOR i ← 0 TO SIZE - 1 DO
   SET TABLE[i] ← EMPTY
END FOR
// 4. INSERT RECORDS INTO HASH TABLE
FOR i ← 0 TO n - 1 DO
   SET index ← A[i] MOD SIZE
   WHILE TABLE[index] ≠ EMPTY DO
      SET index ← (index + 1) MOD SIZE
   END WHILE
   SET TABLE[index] ← A[i]
END FO
// 5. HASH-BASED SEARCH
SET index ← key MOD SIZE
SET start ← index
SET hashResult ← NOT FOUND
WHILE TABLE[index] ≠ EMPTY DO
   IF TABLE[index] = key THEN
      SET hashResult ← index
      BREAK
   END IF
   SET index ← (index + 1) MOD SIZE
```

```
        IF index = start THEN
            BREAK
        END IF
    END WHILE
    // 6. DISPLAY RESULTS
    OUTPUT "LINEAR SEARCH RESULT"
    OUTPUT linearResult
    OUTPUT "BINARY SEARCH RESULT"
    OUTPUT binaryResult
    OUTPUT "HASH SEARCH RESULT"
    OUTPUT hashResult
END
ALGORITHM BINARY-SEARCH(A, low, high, key)
BEGIN
    IF low > high THEN
        RETURN NOT FOUND
    END IF
    SET mid ← low + FLOOR((high - low) / 2)
    IF A[mid] = key THEN
        RETURN mid
    END I
    IF key < A[mid] THEN
        RETURN BINARY-SEARCH(A, low, mid - 1, key)
    ELSE
```

```
        RETURN BINARY-SEARCH(A, mid + 1, high, key)
    END IF
END
```

Example Output

For registration number 412:

LINEAR SEARCH RESULT

3

BINARY SEARCH RESULT

3

HASH SEARCH RESULT

9

EXPLANATION OF THE PSEUDOCODE

- **STUDENT-RECORD-SEARCH:** This is the main algorithm used to search for a student's registration number using three different searching techniques.
- **INPUT:** The algorithm accepts an array of student registration numbers, the total number of records, and the registration number to be searched.
- **Array A:** The array stores the registration numbers of students in ascending order.
- **Number of Records (n):** The variable n stores the total number of student records available in the array.
- **Input Key:** The variable key represents the registration number entered by the user for searching.
- **Initialize Array:** The array is initialized with seven registration numbers: 101, 205, 309, 412, 518, 624, and 735.
- **Calculate Length:** The LENGTH(A) operation determines the total number of elements present in the array.

- **Linear Search:** Linear Search checks each registration number sequentially until the required number is found or all records have been checked.
- **Initialize Linear Result:** The variable linearResult is initially set to NOT FOUND, indicating that the registration number has not yet been located.
- **FOR Loop:** The loop starts from index zero and continues until the last element of the array is reached.
- **Compare Elements:** The condition A[i] = key checks whether the current registration number matches the required registration number.
- **Store Result:** If a match is found, the current array index is stored in linearResult.
- **BREAK Statement:** The BREAK statement immediately terminates the loop because the required registration number has been found.
- **Unsuccessful Search:** If no matching element exists, linearResult remains NOT FOUND.
- **Binary Search:** Binary Search efficiently locates the registration number by repeatedly dividing the sorted array into smaller sections.
- **Initialize Low:** The variable low is set to zero, representing the first index of the array.
- **Initialize High:** The variable high is set to n - 1, representing the last index of the array.
- **Call Binary Search:** The algorithm calls BINARY-SEARCH with the array, starting index, ending index, and required registration number.
- **Initialize Hash Table:** A hash table is created with 101 slots to store student registration numbers.
- **Initialize Table Slots:** Every slot in the hash table is assigned the value EMPTY to indicate that it does not contain a registration number.
- **Insert Records:** Each registration number from the original array is inserted into the hash table.
- **Calculate Hash Index:** The expression A[i] MOD SIZE calculates the initial storage location using the remainder operation.

- **Check Collision:** A collision occurs when two registration numbers map to the same hash-table location.
- **Linear Probing:** If the calculated slot is occupied, the algorithm checks the next slot until an empty location is found.
- **Store Registration Number:** The registration number is inserted into the available hash-table slot.
- **Hash-Based Search:** Hash-Based Search calculates the expected location of the registration number and checks the table for a match.
- **Calculate Search Index:** The expression key MOD SIZE determines the initial location where the required registration number should be stored.
- **Store Starting Index:** The variable start saves the initial location so the algorithm can detect when it has completed a full cycle.
- **Initialize Hash Result:** The variable hashResult is initially assigned NOT FOUND.
- **Check Occupied Slot:** The WHILE loop continues as long as the current hash-table slot is not empty.
- **Compare Registration Number:** The condition TABLE[index] = key checks whether the current slot contains the required registration number.
- **Store Hash Result:** When a match is found, the current hash-table index is stored in hashResult.
- **Terminate Search:** The BREAK statement stops the search immediately after the registration number is found.
- **Check Next Slot:** If the current slot does not contain the required registration number, the algorithm moves to the next slot using (index + 1) MOD SIZE.
- **Detect Full Cycle:** If the search returns to the starting index, the algorithm stops to avoid searching the same slots repeatedly.
- **Display Linear Result:** The algorithm displays the index returned by Linear Search.
- **Display Binary Result:** The algorithm displays the index returned by Binary Search.

- **Display Hash Result:** The algorithm displays the hash-table slot returned by Hash-Based Search.
- **Binary Search Function:** The BINARY-SEARCH algorithm is a separate procedure that recursively searches the sorted array.
- **Check Search Range:** If low > high, the search range is empty, meaning the registration number was not found.
- **Calculate Middle Index:** The expression low + FLOOR((high - low) / 2) calculates the middle index of the current search range.
- **Check Middle Element:** If A[mid] = key, the middle element matches the required registration number, and its index is returned.
- **Search Left Half:** If the required registration number is smaller than the middle element, the algorithm recursively searches the left half.
- **Search Right Half:** If the required registration number is greater than the middle element, the algorithm recursively searches the right half.
- **Repeat Search:** The recursive calls continue reducing the search range until the registration number is found or the range becomes empty.
- **Return Index:** When a registration number is found, its array index is returned to the main algorithm.
- **Return NOT FOUND:** If the registration number does not exist in the array, the algorithm returns NOT FOUND.
- **Example Result:** When the registration number 412 is searched, Linear Search and Binary Search return array index 3.
- **Hash Result:** The hash function calculates 412 MOD 101 = 9, so Hash-Based Search returns slot 9.
- **Conclusion:** The algorithm compares three searching techniques to demonstrate their working methods and efficiency. Linear Search takes O(n) time in the worst case, Binary Search takes O(log n) time, and Hash-Based Search has an expected search time of O(1) under suitable conditions.